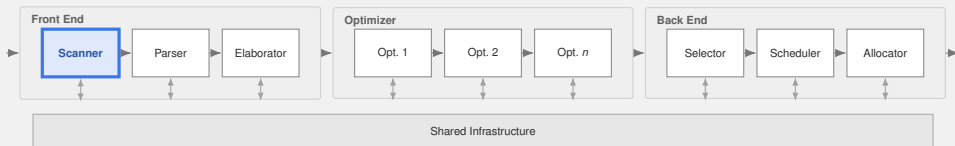


# Scanners

MSc. Gina Muñoz Salas

# Where we are



The scanner is the only pass that touches *every* character of the input program. Its input is larger than any other pass's, and the techniques that make it fast are simple enough to state in one lecture.

# What a scanner does

## Characters in, words out

Each call returns a pair  $\langle \text{lexeme}, \text{category} \rangle$ , sometimes called a **token**. The lexeme is the spelling; the category is the part of speech: a terminal symbol of the grammar the parser will use.

The rules that group characters into words are the language's *microsyntax*.

```
x = count * 2
```

$\langle x, \text{identifier} \rangle$   
 $\langle =, \text{assign} \rangle$   
 $\langle \text{count}, \text{identifier} \rangle$   
 $\langle *, \text{times} \rangle$   
 $\langle 2, \text{number} \rangle$



# Three different times

01

## Design time

The compiler writer specifies the microsyntax: how words are spelled. She also picks an implementation strategy.

02

## Build time

Tools turn that specification into code, and a compiler turns the code into an executable scanner.

03

## Compile time

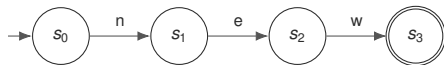
The end user runs the compiler. The scanner converts the application code into a stream of classified words.

01

# Recognizing words

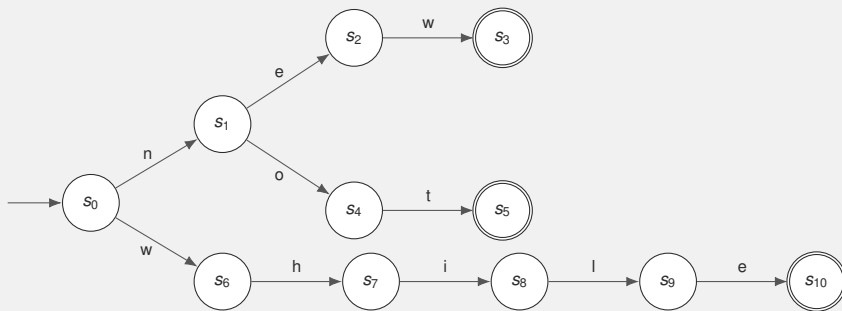
# A recognizer for one word

```
c ← NextChar();  
if (c = 'n') then  
  c ← NextChar();  
  if (c = 'e') then  
    c ← NextChar();  
    if (c = 'w') then  
      report success;  
    else try something else;  
  else try something else;  
else try something else;
```



One test per character. The double circle is an *accepting* state. Transitions to the error state  $s_e$  are implicit and never drawn.

# Recognizers combine



new, not and while in one recognizer: merge the start states, relabel, share common prefixes. Three accepting states, one transition per input character, running time proportional to the length of the input.

# The formalism: finite automaton

## A five-tuple $(S, \Sigma, \delta, s_0, S_A)$

$S$  the finite set of states, including  $s_e$

$\Sigma$  the finite alphabet

$\delta(s, c)$  the transition function

$s_0$  the start state

$S_A \subseteq S$  the accepting states

## Acceptance

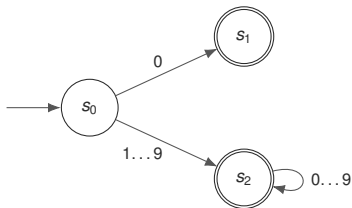
The FA accepts  $x_1 x_2 \dots x_n$  if and only if

$$\delta(\dots \delta(\delta(s_0, x_1), x_2) \dots, x_n) \in S_A$$

Two ways to fail: some  $x_j$  drives the FA into  $s_e$ , or the input runs out in a nonaccepting state. If the FA *passed through* an accepting state on the way, the input contains a valid word as a prefix: the observation scanners use to find word boundaries.



# Infinite languages need cycles



An unsigned integer is a zero, or a nonzero digit followed by zero or more digits. The self-loop replaces an infinite path.

## $\delta$ as a table

| $\delta$ | 0     | 1...9 | other |
|----------|-------|-------|-------|
| $s_0$    | $s_1$ | $s_2$ | $s_e$ |
| $s_1$    | $s_e$ | $s_e$ | $s_e$ |
| $s_2$    | $s_2$ | $s_2$ | $s_e$ |

## The skeleton never changes

```
state  $\leftarrow s_0$ ;  
char  $\leftarrow$  NextChar();  
while (state  $\neq s_e$  and char  $\neq$  eof) do  
    state  $\leftarrow \delta(\text{state}, \text{char})$ ;  
    char  $\leftarrow$  NextChar();  
if (state  $\in S_A$ ) then accept; else reject;
```

# In class

## EXERCISE

### Build two recognizers

1. An FA for identifiers: one alphabetic character followed by *up to five* alphanumeric characters.
2. An FA for a Pascal comment: an open brace, then zero or more characters drawn from  $\Sigma - \{\}\}$ , then a close brace.

How many states does each need? Which one needs a cycle, and why?

02

## Regular expressions

# Three operations, and nothing else

01

## Alternation

$r \mid s$  denotes  $L(r) \cup L(s)$ .

bow | row, or equivalently  
(b|r)ow.

02

## Concatenation

$rs$  denotes  $\{xy \mid x \in L(r), y \in L(s)\}$ .

Writing symbols side by side  
means: in this order.

03

## Kleene closure

$r^* = \bigcup_{i=0}^{\infty} r^i$ : zero or more words  
from  $L(r)$ .

This is the notation for the FA's  
cyclic edge.

Precedence: parentheses, then closure, then concatenation, then alternation. The positive closure  $r^+ = rr^*$  and the finite closure  $r^i$  are conveniences, both rewrite into the three basic operations, so we ignore them in the constructions.

# Microsyntax of a real language

|                  |                                                                               |
|------------------|-------------------------------------------------------------------------------|
| Identifier       | $([A\dots Z] \mid [a\dots z]) ([A\dots Z] \mid [a\dots z] \mid [0\dots 9])^*$ |
| Unsigned integer | $0 \mid [1\dots 9] [0\dots 9]^*$                                              |
| Real number      | $(0 \mid [1\dots 9] [0\dots 9]^*) (\epsilon \mid . [0\dots 9]^*)$             |
| Character string | $" (\wedge)^* "$                                                              |
| Line comment     | $// (\wedge \backslash \texttt{n})^* \backslash \texttt{n}$                   |
| Block comment    | $/* (\wedge^* \mid *^+ \wedge /)^* */$                                        |
| Keyword          | <code>if while return</code>                                                  |

$\wedge c$  is the complement of  $c$  with respect to  $\Sigma$ , the operator `lex` and `flex` spell  $\wedge$ . Note how much of the complexity in the block-comment RE comes from a *two-character* delimiter.

# More states cost space, not time

## Loose specification

$r[0 \dots 9]^+$

Two states. Accepts `r29`. Also accepts `r999999`, although no processor has 100,000 registers.

## Tight specification

$r([0 \dots 2]([0 \dots 9]|\epsilon) | [4 \dots 9] | (3(0|1|\epsilon)))$

More states, more transitions, more memory. Accepts `r0`, `r00`, `r31`; rejects `r32`.

**The cost of running an FA is proportional to the length of the input**, not to the complexity of the RE or the number of states. Both automata above take exactly one transition per character. Build time may grow; run time does not.

# In class

## EXERCISE

### Rewrite and design

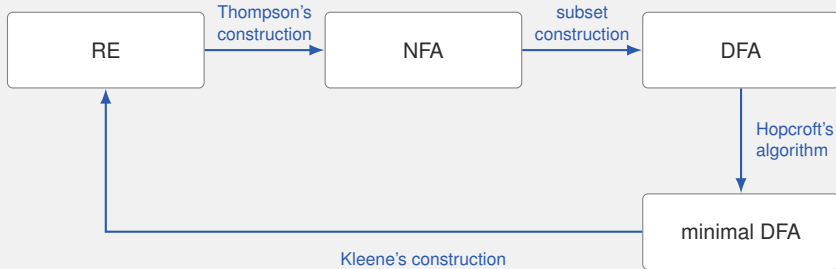
1. A six-character identifier is often written with a finite closure:  $([A \dots Z] \mid [a \dots z])([A \dots Z] \mid [a \dots z] \mid [0 \dots 9])^5$ . Rewrite it using only alternation, concatenation and Kleene closure.
2. In PL/I a string starts and ends with ", and a literal quotation mark is written by doubling it. Give an RE and an FA that recognize PL/I strings.

03

**From RE to DFA**



# The cycle of constructions

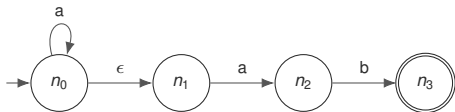


The cycle proves that REs and FAs have equivalent expressive power: anything an FA can recognize, an RE can specify, and vice versa. We will walk the first three arrows; Kleene's construction is in §2.6.1.

# Nondeterminism

## Where it comes from

An  $\epsilon$ -transition consumes no input. It lets us glue automata together — but it can give a state two ways out on the same character.



On a in  $n_0$ : loop, or step through  $\epsilon$ ? For aab the first is right; for ab, the second.

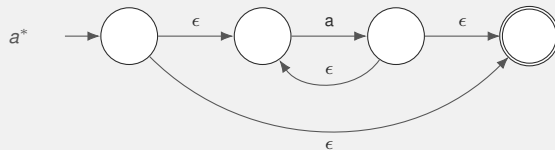
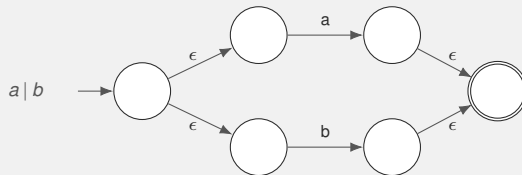
## Two models of NFA behavior

**Omniscient.** At each choice, the NFA guesses the transition that leads to acceptance, if one exists.

**Cloning.** At each choice, the NFA clones itself and pursues every path at once. The set of simultaneously active states is its *configuration*.

An NFA with  $N$  states has at most  $|2^N|$  configurations, a finite number. That is exactly why a DFA can simulate it, at the price of space but not of time.

# Thompson's construction



Template-driven: build a trivial NFA for each symbol, then apply one transformation per operator, in precedence order. Every NFA produced has *one* start state and *one* accepting state, nothing enters the start state, nothing leaves the accepting state — properties that make the next construction easy to implement.

# The subset construction

```
q0 ← FollowEpsilon({n0})
Q ← q0
WorkList ← {q0}
while (WorkList ≠ ∅) do
  remove q from WorkList
  for each character c ∈ Σ do
    temp ← FollowEpsilon(Delta(q,c))
    if temp ≠ ∅ then
      if temp ∉ Q then
        add temp to Q and WorkList
      T[q,c] ← temp
```

## What it builds

$Q$  holds every *valid configuration* the NFA can reach;  $T$  holds the transitions between them. One DFA state per configuration, one DFA transition per entry of  $T$ .

A configuration  $q_i$  is accepting iff it contains an accepting state of the NFA.

$Q$  can grow to  $|2^N|$  states. It never shrinks, and that monotonicity is what guarantees the loop halts.

# A fixed-point computation

01

## Monotone function

Each pass over the worklist only *adds* to  $Q$ :  $Q_i \subseteq Q_{i+1}$ . Nothing is ever removed.

02

## Bounded domain

Every  $q_i$  is a subset of  $N$ . Since  $N$  is finite,  $2^N$  is finite, so the loop cannot iterate forever.

03

## Unique result

Set union is commutative and associative, so the order of removal from the worklist cannot change the final  $Q$ : only the names of its elements.

This pattern, iterate a monotone function over a bounded domain until nothing changes, will return in data-flow analysis, in FIRST/FOLLOW sets, and in register allocation. Recognize it now and you will recognize it four more times this semester.

# DFA minimization

## Hopcroft's algorithm

Build a partition  $P = \{p_1, \dots, p_m\}$  of the DFA states, where each  $p_i$  is a set of *behaviorally equivalent* states.

Start with the coarsest split  $P_0 = \{D_A, D - D_A\}$ , then refine until no set can be split.

Hopcroft's algorithm only ever splits sets; it never merges them.

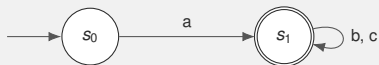
## The two rules

1. If  $d_i, d_j$  are in the same set and  $d_i \xrightarrow{c} d_x$ ,  $d_j \xrightarrow{c} d_y$ , then  $d_x$  and  $d_y$  must also land in the same set — for every  $c \in \Sigma$ .
2. A set contains either accepting states or nonaccepting states, never both.

Fewer states means a smaller table, which may fit in the first-level cache. On modern machines that can matter more than the transition count.

# The whole pipeline on $a(b \mid c)^*$

## DFA after minimization



Thompson's construction produces an NFA with ten states. The subset construction turns it into a four-state DFA. Minimization collapses it to two. **This is the automaton a human would have drawn by hand**, which is the point: the mechanical path arrives at the same place, without requiring insight.

# In class

## EXERCISE

### Run the pipeline

Take the RE `who | what | where.`

1. Apply Thompson's construction to build an NFA.
2. Apply the subset construction to build a DFA.
3. Minimize it.

How many states does each stage have? Where does the sharing of the common prefix `wh` finally appear?



04

**From DFA to scanner**

# A scanner is not a DFA

## Different model of execution

A DFA reads *all* its input and answers yes or no. A scanner reads *enough* input to find the next word, and leaves the stream positioned to find the one after it.

So the scanner runs until it hits  $s_e$ , and then **rolls back**, one character at a time, to the most recent accepting state.

It must therefore record the states it passed through, or invert  $\delta$ .

## Different answer

A DFA returns a bit. A scanner returns  $\langle \text{lexeme}, \text{category} \rangle$ .

So each accepting state must map to exactly one category. Build a separate NFA per rule, join them with  $\epsilon$ -transitions from a new start state, and let the subset construction do the work.

When two rules overlap then matches both the keyword rule and the identifier rule, the generator resolves it by precedence. `lex` and `flex` give priority to the rule listed first.

# Two ways to handle keywords

01

## One rule per keyword

The scanner returns the category through the same mechanism it uses for numbers and identifiers. The DFA gets more states and generation costs a little more, but it is still  $O(1)$  per character.

02

## Preload the symbol table

Fold keywords into the identifier rule, and preload the table of identifier names with the keywords and their categories. The scanner then finds each keyword as an already-categorized identifier.

Option 02 comes almost free: most scanners build that table anyway, both as a start on the compiler's symbol table and as a way to map names to small integers for cheap comparison.

# Whitespace decides more than it looks

01

## Most languages

Whitespace has no meaning; the scanner discards it. Its real effect is its *absence* from the REs, that is what separates `do` from `i`, and makes `a * b` scan like `a*b`.

02

## Fortran 66

Blanks are not significant at all. To tell `do 10 i = 1` from `do 10 i = 1,100`, the scanner must read past the `=` to the comma. Few languages repeated the experiment.

03

## Python

The opposite extreme: the number of leading blanks determines block structure. Add a rule that matches an end-of-line plus zero or more blanks, then compare its length with the previous one.

**ChocoPy inherits Python's rule.** Your scanner will emit `INDENT` and `DEDENT` as words that appear nowhere in the source text, keep an indentation stack, and remember that a single dedent may need to close several blocks at once.

**05**

## **Implementing scanners**

# Three implementation strategies

01

## Table-driven

A fixed skeleton plus generated tables:  $\delta$  for transitions, `CharClass` to map a character to a column. Language knowledge lives entirely in the data.

02

## Direct-coded

The same DFA, represented *implicitly*: one custom code fragment per state, transitions become jumps. No address arithmetic, no memory loads per character.

03

## Hand-coded

Written by the compiler writer. Allows specializing individual paths, dropping the state stack when there is one accepting state, and returning a register *number* rather than a string.

All three are  $O(1)$  per character; they differ in the constants. Hand-coded scanners remain surprisingly common in production compilers, GCC 4.0 used them in several front ends, despite `flex` having been built for that project.

# Where the time goes

## The table-driven inner loop

```
while (state  $\neq$   $s_e$ ) do
  char  $\leftarrow$  NextChar();
  col  $\leftarrow$  CharClass[char];
  state  $\leftarrow$   $\delta$ [state, col];
```

Two address computations and two loads per character:

$@CharClass_0 + i \times w$

$@\delta_0 + (state \times cols + col) \times w$

## What direct coding removes

Each state becomes a labelled fragment. char, lexeme and the implicit state all fit in registers; the branch replaces the table lookup entirely.

```
s1: char  $\leftarrow$  NextChar();
    lexeme  $\leftarrow$  lexeme + char;
    push(s1);
    if ('0'  $\leq$  char  $\leq$  '9')
      then goto s2;
      else goto sout;
```

# Maximal munch and quadratic roll back

## The pathological case

Take  $ab \mid (ab)^*c$  and the input `abababab`.

The scanner reads all eight characters before it can conclude the longest valid prefix is `ab`. The next call reads six, then four, then two  $O(n^2)$  in the worst case.

The cause is the desire to return the *longest* word that is a prefix of the input.

## The fix

Keep a bit array `Failed[state, position]`. Each time roll back pops a dead-end transition, mark it. Before taking a transition, test the bit and break out if it is already known to fail.

Most programming languages have microsyntax simple enough that this never arises, but the cost of insuring against it is one bit test per character.



# Two practical details

## Buffering the input

One function call per character is expensive relative to simulating the DFA. Read into a buffer instead and index through it; `RollBack` becomes a pointer decrement.

Use **two adjacent buffers** of  $n$  characters and increment modulo  $2n$ . A `Fence` pointer marks how far back it is safe to go.

With  $n = 2048$  or  $4096$ , roll back is bounded by the longest whitespace-free string in the program. Few identifiers are 2048 characters long.

## Compressing the table

$\delta$  has  $|\text{states}| \cdot |\Sigma|$  entries. If it outgrows the first-level cache, performance suffers.

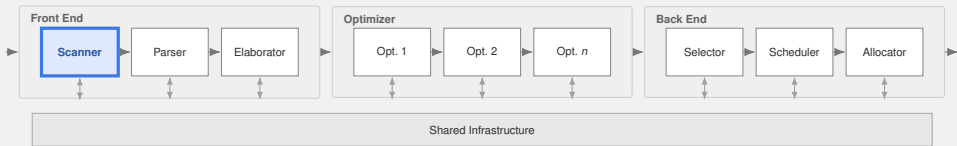
Identical *columns* are common, the ten digit columns of an integer recognizer are the same. Merge them and map characters to columns through `CharClass`.

If the DFA is minimal, its *rows* cannot be identical, that is exactly what minimization guarantees. And the row for  $s_e$  is never read, so it need not be stored.

# Summary

The compiler writer specifies microsyntax with **regular expressions**. Thompson's construction turns them into an **NFA**, the subset construction into a **DFA**, Hopcroft's algorithm into a **minimal DFA**, and a skeleton or a code generator into a **scanner**.

Every step is mechanical. That is the whole point: the scanner is the one part of the compiler where theory delivers a complete, automatic construction, and where the resulting program runs in  $O(1)$  time per character regardless of how complicated the specification was.



# References

## **Engineering a Compiler, 3rd ed.**

Cooper & Torczon, 2023.

Chapter 2: Scanners.

§2.2 recognizers · §2.3 regular expressions · §2.4 RE to DFA · §2.5 implementation · §2.6 advanced topics.

## **The Dragon Book**

Aho, Lam, Sethi & Ullman.

Chapter 3: Lexical Analysis.

Complementary treatment of the same constructions, with more detail on `lex`-style specifications.

# Scanners

MSc. Gina Muñoz Salas